Effects of Varying Uncertainty Measures on Bayesian SegNet

Adam Sigal
ECSE 626
adam.sigal@mail.mcgill.ca

Abstract

Semantic segmentation is a crucial tool in understanding visual input. All systems which must navigate through, interact with, or simply parse real-world scenes must be able to break them down into their major parts. However, many of the top-performing semantic segmentation algorithms fail to account for any uncertainty in their predictions. Kendall, et al. [13] propose an approach that produces a probabilistic segmentation with a measure of uncertainty on image data. They hypothesize that considering uncertainty will not only improve the performance of the model, but is intrinsically important for the task of semantic segmentation. Kendall, et al. base their approach on their earlier architecture, SegNet [1], to create Bayesian SegNet, a probabilistic CNN encoder-decoder framework for pixel-wise semantic segmentation. The architecture employs dropout at test time to approximate the posterior distribution by sampling from the Bernoulli distribution across the network's weights. According work by to Gal and Ghahramani [9], dropout can approximate Bayesian inference over the weights of a Bayesian CNN. In this work, I reimplemented Bayesian SegNet in PyTorch and explored the use of different measures of uncertainty: variance, entropy, and mutual information.

1. Introduction

Semantic segmentation is a powerful tool for image understanding in computer vision. It describes the process of assigning a class label to every pixel in a given image. In a perfect scenario, this process allows a system to understand the contents, geometry, and object relationships present in visual input with great precision. Automatic understanding of images is applicable to domains such as autonomous driving, robotics, manufacturing, and medical imagery. However, doing this task successfully poses many problems. For example, images with the same or similar content can appear very different visually. Many factors influence this, such as lighting, angle, obstructions, camera type, and more. To approach these issues, there have been a diverse array of methods proposed. This is the task that Kendall *et al*. aim to further in [13]. The present work is a reproduction, analysis, and extension of the methods and results of that study.

Earlier segmentation approaches used low-level and intrinsic features of pixel data [18]. More recently, deep learning methods have set the benchmark on many datasets for semantic segmentation tasks [14]. One issue which was present in all of these approaches prior to the publication of Kendall *et al*.'s study, was the lack of a measure of uncertainty allowing for image segmentation with probabilistic considerations. The authors argue that uncertainty is crucial in order to have a meaningful understanding of the information gleaned from segmentation. It is possible that a system may not have chosen a label for a certain object with great confidence, and thus its output should judged accordingly. This is especially true in safety-critical and otherwise widely deployed systems. Humans have a predisposition to assign trust to automated systems, and therefore it is imperative that uncertainty in such systems be not only available, but implemented in their end use-cases, and communicated clearly.

The proposed approach leverages Bayesian convolutional neural networks (Bayesian CNNs) in order to add a measure of uncertainty to semantic segmentation machine learning methods. This structure produces a pixel-wise probabilistic segmentation output. The authors call this architecture Bayesian SegNet, as it is based on their previous work, SegNet [1]. In Bayesian SegNet, the authors apply findings by Gal and Ghahramani [10] demonstrating the ability of dropout layers to approximate the posterior distribution over the weights of a Bayesian CNN. These findings permitted the authors to measure the uncertainty present in their model in a clear way.

Bayesian CNNs are part of a larger class known as Bayesian neural networks [6, 15]. Put simply, these are neural networks that model the uncertainty of their predictions by inferring distributions over their weights. While theoretically important, at the time that Kendall *et al*.'s study was published, Bayesian NNs showed little improvement in performance compared to corresponding non-Bayesian meth-

ods, while conversely coming at the expense of significant computational complexity. Furthermore such models had been observed to be subject to overfitting, further decreasing their utility. In fact, some theoretical Bayesian methods are computationally infeasible in practice, and therefore approximations must be used. This will be discussed further in Section 2.

2. Methods

2.1. Architecture

SegNet is a machine learning model following a *deep convolutional encoder-decoder* architecture (Fig. 2). The model is split into *encoding units*: a group of convolutional layers, each followed by a batch normalization layer with ReLU activations, with a max pooling layer at the very end, and *decoding units*: an upsampling layer, similarly followed by a group of convolutional layers with batch normalization and ReLU after each. The model uses 13 encoding units and 11 corresponding dropout units. The indices from the max pooling layers are saved for use in the corresponding upsampling layers, allowing for class boundary information to be carried forward. The 13 encoder units use the same structure as VGG-16 [21], allowing for transfer learning from the easily accessible pre-trained weights of VGG-16 to SegNet. Kendall *et al*. also use SegNet-Basic (architecture also found in [1]), a smaller variant with 4 layers of encoder units and 4 layers of decoder units. This was used for comparative analysis of Bayesian SegNet architectures (Section 2.3). The authors posit that this architecture is able to approximate the learning behaviour of the full-sized SegNet, while saving on training time.

Kendall *et al*. implemented their models in Caffe [12] and their code is publicly available here [13]. The first main contribution of the present work is to reimplement Bayesian SegNet into the widely used PyTorch framework.

I planned to build off of a preimplemented base SegNet model. In my search I considered using a handful of publicly available versions of SegNet: [5, 20, 11, 17]. The chosed base was that built by Goswami [11] as it was the most cleanly implemented. On top of this, I built the Bayesian SegNet architecture and training / testing pipeline.

2.2. Dropout as Bayesian Approximation

In building Bayesian SegNet, Kendall *et al*. used dropout layers to approximate inference on the posterior distribution over the model. That is to say, uncertainty is found by performing variational inference with Bernoulli distributions over the network's weights. This is based on work by Gal and Ghahramani [10] which showed that dropout can approximate Bayesian inference at test time. This is in contrast to dropout's normal usage as a regularization tool, where it is deactivated at test time.

Using dropout at test time causes variations in the output depending on the random set of weights of the convolutional kernels that stay activated. The rate of dropping out can in theory be optimized, but the authors fixed this value at $p_i = 0.5$, and so I did the same in my implementation. Running a forward pass through the network multiple times with different weights activated by dropout gives us a range of outputs from which we can glean a measure of uncertainty. In this case, the variance between the samples is used. The authors also tested using *variation ratio* (percentage of predictions that agree with chosen class prediction), but this was found to give a less robust measure of uncertainty. This process is called Monte-Carlo sampling, and the authors performed 10 per input image during training and testing.

The extent to which the amount of monte-carlo samples would enhance the results and performance of Bayesian SegNet was also explored. This will be discussed further in Section 4.

Following the findings of Gal and Ghahramani, these samples are approximately equivalent to samples from the posterior distribution of the model's weights W: $p(W|X, Y)$, where X is a training image and Y is its segmentation mask. This posterior in reality is intractable, hence the approximation. If we can minimize the KL divergence between an approximation $q(W)$ of the posterior distribution of the weights and its actual value,

$$KL(q(W)||p(W|X, Y)) \tag{1}$$

then we can use the approximation instead with validity. The approximate distribution $q(W)$ for each $K \times K$ dimensional convolutional layer follows a Bernoulli distribution at each of its units:

$$b_{i,j} \sim \text{Bernoulli}(p_i) \quad \text{for } j \in \{1, \ldots, K\} \tag{2}$$

$$W_i = M_i \, \text{diag}(b_i) \tag{3}$$

where b_i is number of vectors of Bernoulli distributed random variables, M_i contains the variational parameters, and p_i is dropout probability.

Together, this gives us an approximate Gaussian process, as long as there are dropout layers between every convolutional layer. However, this is notably not the case in Kendall *et al*.'s study. This is discussed further in Section 2.3 and 4.

In [9], the authors prove that minimizing the cross-entropy loss of such a Bayesian CNN has the effect of minimizing the KL divergence described in (1). Therefore, training the network with gradient descent to minimize the cross-entropy loss will minimize the KL divergence as well, thus giving us a valid approximation of the posterior.

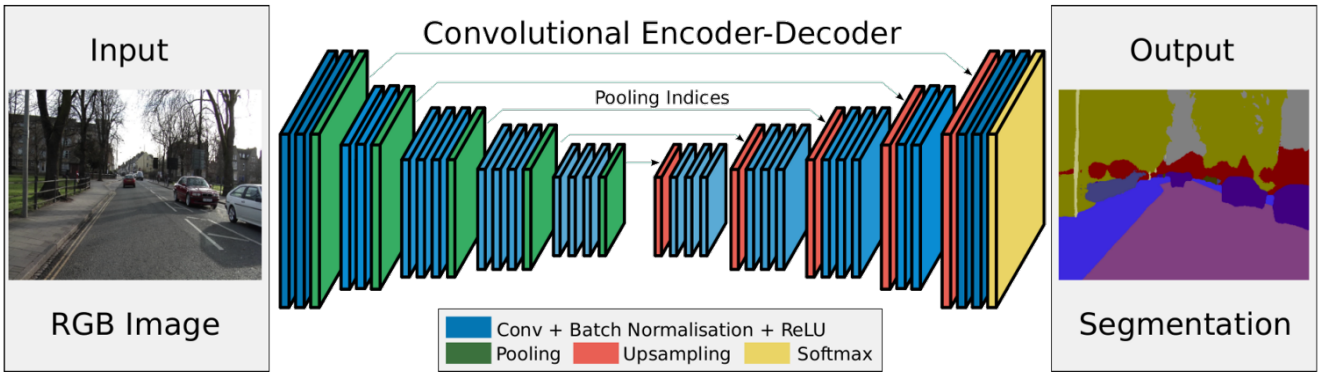


Figure 1: SegNet architecture

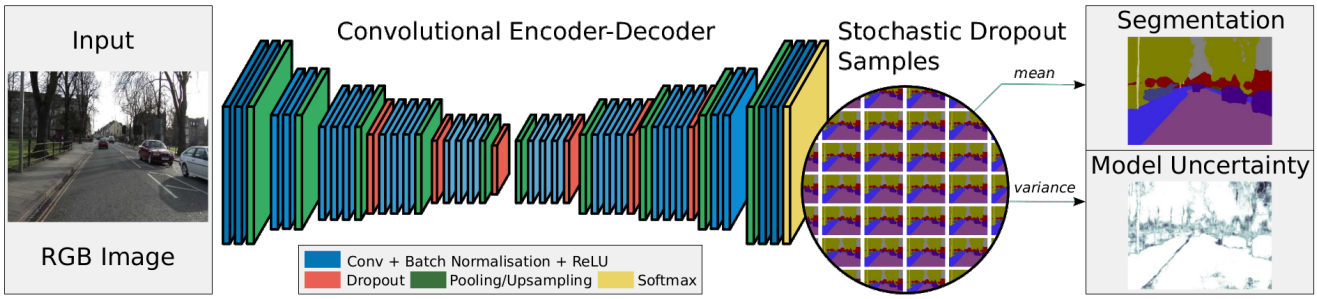


Figure 2: Bayesian SegNet architecture

2.3. Probabilistic Variants

The variational inference process described in Section 2.2 was proven with the assumption that there are dropout layers after every convolutional layer. Empirically, Kendall *et al*. found this to be too strong of a regularizer at training time. Therefore, the authors trained and compared six architectures with different dropout layer placements, all based on SegNet-Basic (see Section 4.1 of [13]).

The best empirical placement of dropout layers was found to be after the last three encoding units and first three decoding units (Fig. 1). The authors propose that this result is due to the initial encoders and decoders processing low-level information that is universal to all of the images in the dataset (i.e. edges, corners) and thus it is more useful to activate the weights deterministically in order to perform inference solely on the higher-level image information present in the deeper units.

2.4. Dataset

Kendall *et al*. performed inference on images from three datasets: SUN-RGB-D Indoor Scene Understanding [4], and Pascal VOC12 [8], CamVid Road Scenes [2]. I did not use any of these, but I will briefly describe them and the Kendall *et al*.'s findings using them. SUN-RGB-D Indoor Scene Understanding is a challenging dataset for segmentation because the indoor scenes are highly diverse, with many cases of occlusion and sub-optimal lighting conditions. Bayesian SegNet outperformed both SegNet and all other models against it was tested on this dataset. Pascal VOC12 is another dataset for object segmentation, having varied backgrounds. Bayesian SegNet did not outperform other models on this dataset, but Bayesian methods were shown to increase performance on all models tested. CamVid is a road scene dataset with 367 training images and 233 testing images, and was the principal dataset used in the paper. Bayesian SegNet performed very well on this dataset, with significant improvements on infrequently seen classes. For training they downsampled the images to 360x480 pixels.

On the recommendation of my professor, Dr. Tal Arbel, I used the CityScapes Dataset for training and inference on my implementation of Bayesian SegNet [3]. This dataset is the current standard for segmentation tasks on road scenes, and was even used by the same authors of follow-up work. The version I used contains 2975 training images and 500 validation images, and the dataset contains 34 labels (Table 1). I trained SegNet on images of size 360x480, but for Bayesian SegNet they were further downsampled to 128x256 due to computational costs. This is further discussed in Section 4.

2.5. Training

SegNet was trained by Kendall using stochastic gradient descent (SGD) with cross-entropy loss, using a learning rate of 0.1 and momentum of 0.9. I opted to use mini-batches in an attempt to speed up learning, and used an Adam optimizer with learning rate of 1e-6 and cross-entropy loss. I chose to instead use an Adam optimizer as it has many advantages compared to SGD, such as an adaptive learning rate.

Training one minibatch of 8 images on SegNet took roughly 52 seconds per epoch. Kendall *et al*. trained the model *"to convergence"*, which they said took "over 100" epochs. I trained SegNet for 110 epochs, and it did not converge. This will be discussed further in Section 3. Due to the Monte-Carlo sampling, more GPU memory was needed per mini-batch, so I had to decrease the batch size from 8 to 2 images. Still, training was very slow; roughly 9 minutes per epoch. I was able to train it for 118 epochs, although this meant that it was actually only trained on 236 images, rather than the 880 of SegNet. This likely negatively affected the degree to which the performance of the two models can be directly compared.

Kendall *et al*. used a method called *median frequency class balancing* [7], (eq. (4), (5)), in order to weight each class equally, agnostic of their frequency in the dataset.

$$\alpha_c = \frac{\text{median frequency}}{\text{freq(c)}} \tag{4}$$

$$\text{freq(c)} = \frac{\text{\# pixels of class c in dataset}}{\text{\# pixels in images where c is present}} \tag{5}$$

Some classes are vastly overrepresented when compared with others (for example the *road* class when compared to the *pole* class, see Table 1). I did not have enough time to implement this, which likely also affected my results. This will be discussed further in Sections 3 and 4.

3. Results

The results of the training and validation processes are shown in Figs. 3, 4. Qualitative results on CityScapes are presented in Fig. 5. A comparison between the segmentation and uncertainty results of Kendall *et al*.'s original work and my work are presented in Fig. 6. Quantitative accuracy results of SegNet and Bayesian SegNet on CityScapes are shown in Table 1 in the appendix.

4. Discussion

While in Gal and Ghahramani's original work [9] casting dropout as Bayesian inference in CNNs, 10 Monte-Carlo samples per image are used - I had hoped to study the effects of performing more. This would inevitably lead to a more robust approximation of the uncertainty gleaned from the posterior distribution of the weights. However, the Monte-Carlo sampling proved to be one of the largest limiting factors in computation speed and GPU memory usage. The time to train Bayesian SegNet on a single image when performing 10 Monte-Carlo samples with dropout is roughly 273 seconds, whereas doing the same on SegNet without without the use of sampling takes roughly 6.5 seconds; a 42x decrease.

It is worth noting that Kendall *et al*. affirm that they are able to run SegNet at 35ms per image, and to run Bayesian SegNet at 90ms per image. Furthermore, they used images of roughly double the resolution that I did. I am using an Nvidia RTX 3080 GPU, while Kendall *et al*. used a Titan X GPU. From a cursory internet search, the titan X is less powerful than the RTX 3080. These results indicate that the authors' code is significantly more optimized than mine.

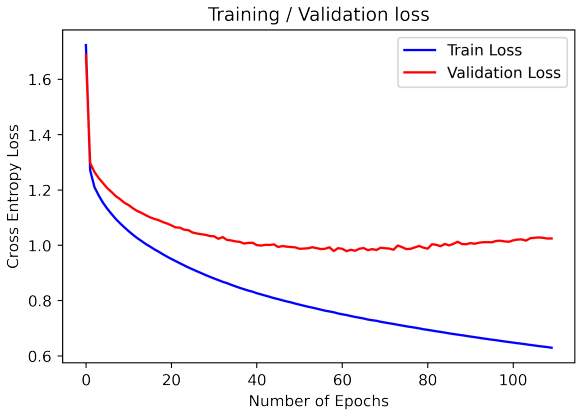


Figure 3: SegNet training and validation cross-entropy loss. Loss is normalized by number of pixels.

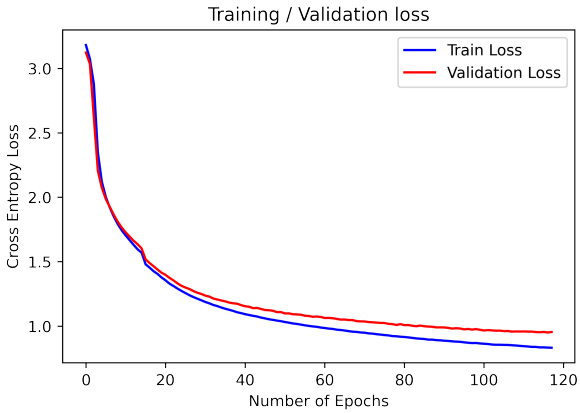


Figure 4: Bayesian SegNet training and validation cross-entropy loss. Loss is normalized by number of pixels.

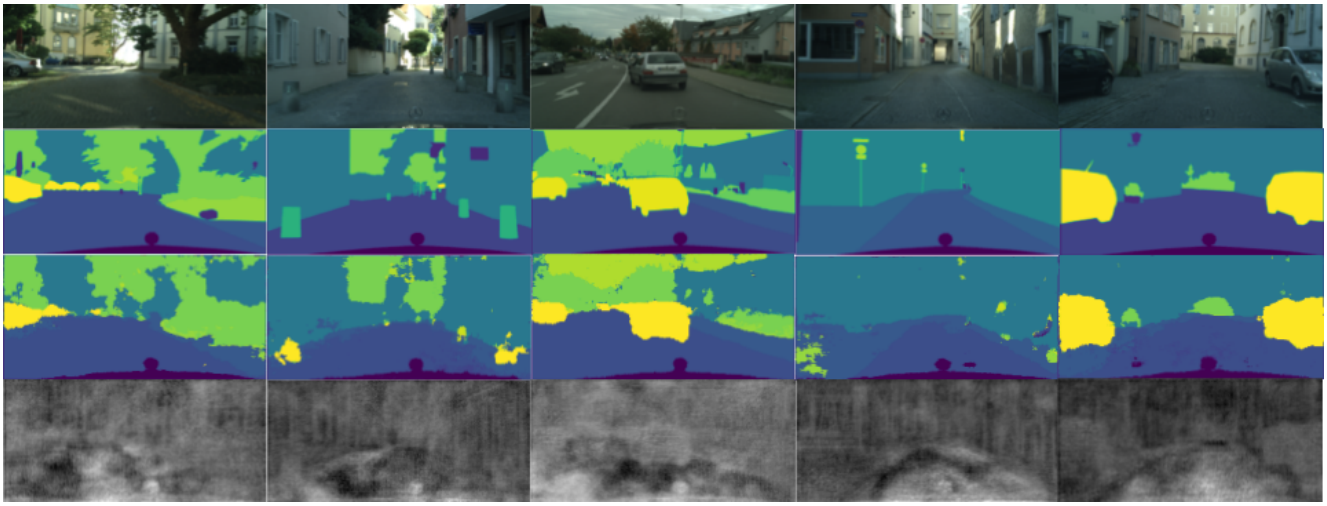
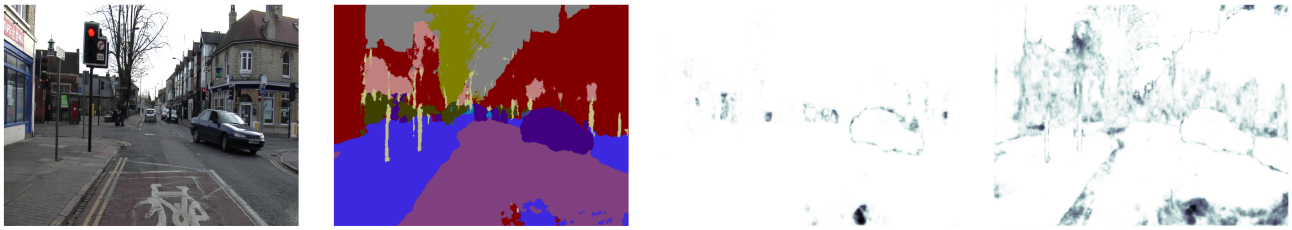
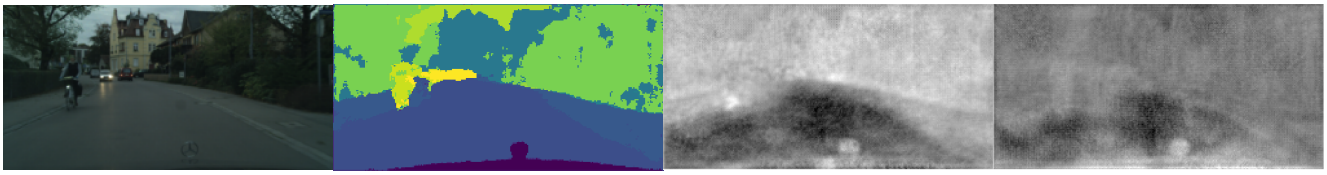


Figure 5: Qualitative Bayesian SegNet segmentation results on the CityScapes dataset. The first row is the input image, the second row is the ground truth segmentation, the third row is Bayesian SegNet's predicted segmentation, and the fourth row is the model uncertainty.



(a) Input Image (b) Semantic Segmentation (c) Uncertainty Car Class (d) Uncertainty All Classes



(e) Input Image (f) Semantic Segmentation (g) Uncertainty Car Class (h) Uncertainty All Classes

Figure 6: Comparison of dropout uncertainty from Monte-Carlo sampling between Kendall *et al.*'s and my implementation. (a) to (d) are Kendall *et al.*'s results on CamVid, and (e) to (h) are my results on CityScapes.

This is one limitation to the scope of a course project and indicates further work is needed to attain more conclusive results.

The last way in which I had planned to extend Kendall *et al.*'s work was to try out and compare more measures of uncertainty. The authors used variance between Monte-Carlo samples as the measure of uncertainty, and also tried using variation ratio (see Section 2.2). In addition, I implemented a measure of pixel-wise entropy between the samples and mutual information. These measures were found to give similar qualitative and quantitative results to those using variance as an uncertainty measure, but in future work I plan to explore these differences further.

In Fig. 3, the loss plot of SegNet's training shows clear signs of overfitting. Quite early on, the training loss begins to descent quickly while the validation loss plateaus and even begins to increase. A solution to this would be the implementation of regularization methods. In fact, Kendall *et al.* say that they used dropout layers for this purpose in their initial SegNet paper [1], however, it was not present in the diagram (Fig. 2). The use of dropout layers in SegNet was also not mentioned in the the Bayesian SegNet paper [13]. This is conspicuous, and the true answer likely lies in the study's official repository (`https://github.com/`

alexgkendall/SegNet-Tutorial), however being unfamiliar with Caffe, I was unable to ascertain what the official approach was.

Bayesian SegNet was trained on 118 epochs to SegNet's 110, however, Bayesian SegNet was trained with batches of only 4 to SegNet's 8. This means that Bayesian SegNet was trained on 4x fewer images than SegNet. Thus, the trend we see in Fig. 4 may be indicative of similar overfitting. However, dropout is a regularizer, and the addition of 6 dropout layers is likely to have a significant effect to combat overfitting. Ultimately to know for certain, more training would have to have been done.

However, looking at Table 1, we see that the performances of both models are very similar. Perhaps Bayesian SegNet is truly better since it achieved this by training over only a quarter of the images, but from these data alone I cannot draw any conclusions. Of note is the low class average accuracy when compared to the global accuracy. This reflects that a small number of classes dominate the total pixel counts (e.g. *road*), while classes that are seldom seen sometimes have a prediction accuracy of zero (e.g. *Train*).

The accuracy results cannot be directly compared with those found by Kendall *et al*., as they used different datasets. The closest one to CityScapes was the CamVid dataset, which only has 11 classes to CityScapes' 33. This alone may have contributed greatly to the overall higher uncertainty seen in my implementation of Bayesian SegNet (see Fig. 6). Indeed, in this figure we can see that uncertainty is overall significantly higher than that found by Kendall *et al*. This indicates the need for further work to ascertain more conclusive results.

In Fig. 5, we see that general shapes of larger objects are overall discernibly segmented. This is qualitatively only slightly worse than the segmentations found in Kendall *et al*.'s work. While the uncertainties are high across the board, they are visibly highest at class boundaries, which reflects the findings of the original authors. This is due to the intrinsic uncertainty present between the boundaries of two objects in any given image.

4.1. Conclusion

In this work I reimplemented SegNet and Bayesian SegNet by Kendall *et al*. in PyTorch, starting from an architecture base by Goswami [11] and explored their performances using different measures of uncertainty. Ultimately, the performance of SegNet and Bayesian SegNet were found to be roughly equivalent, contrary to the findings of the original authors. This is may have been due to overfitting, and thus further work is needed. Variance as a measure of uncertainty appears to be supported due to its higher values around class boundaries in the predicted segmentation mask, however uncertainty in these models was universally quite high. Future work includes a deeper look into the different types of uncertainty measures, and the effect of a high monte-carlo dropout sampling rate on performance.

References

[1] V. Badrinarayanan, A. Kendall, and R. Cipolla. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 39(12):2481–2495, 2017.

[2] G. J. Brostow, J. Fauqueur, and R. Cipolla. Semantic object classes in video: A high-definition ground truth database. *Pattern Recognition Letters*, 30(2):88–97, 2009.

[3] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] C. Couprie, C. Farabet, L. Najman, and Y. LeCun. Indoor semantic segmentation using depth information. *arXiv preprint arXiv:1301.3572*, 2013.

[5] Delta Onera. Segnet pytorch. https://github.com/delta-onera/segnet_pytorch, 2017.

[6] J. S. Denker and Y. LeCun. Transforming neural-net output levels to probability distributions. In *Proceedings of the 3rd International Conference on Neural Information Processing Systems*, pages 853–859, 1990.

[7] D. Eigen and R. Fergus. Predicting depth, surface normals and semantic labels with a common multi-scale convolutional architecture. In *Proceedings of the IEEE international conference on computer vision*, pages 2650–2658, 2015.

[8] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The PASCAL Visual Object Classes Challenge 2012 (VOC2012) Results. http://www.pascal-network.org/challenges/VOC/voc2012/workshop/index.html.

[9] Y. Gal and Z. Ghahramani. Bayesian convolutional neural networks with bernoulli approximate variational inference. *arXiv preprint arXiv:1506.02158*, 2015.

[10] Y. Gal and Z. Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059. PMLR, 2016.

[11] S. Goswami. pytorch-segnet. https://github.com/say4n/pytorch-segnet, 2020.

[12] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv preprint arXiv:1408.5093*, 2014.

[13] A. Kendall, V. Badrinarayanan, and R. Cipolla. Bayesian segnet: Model uncertainty in deep convolutional encoder-decoder architectures for scene understanding. In *British Machine Vision Conference 2017, BMVC 2017*, 2017.

[14] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. *CoRR*, abs/1411.4038, 2014.

[15] D. J. MacKay. A practical bayesian framework for backpropagation networks. *Neural computation*, 4(3):448–472, 1992.

[16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.

[17] Python Software Foundation. pytorchcv. `https://pypi.org/project/pytorchcv/`, 2021.

[18] J. Shotton, J. Winn, C. Rother, and A. Criminisi. Textonboost for image understanding: Multi-class object recognition and segmentation by jointly modeling texture, layout, and context. *International Journal of Computer Vision*, 81:2–23, 01 2009.

[19] S. A. D. Yassine Ouali, Inwan Yoo. Semantic segmentation in pytorch. `https://github.com/yassouali/pytorch-segmentation`, 2021.

[20] W. Zhang. Semantic segmentation in pytorch. `https://github.com/Charmve/Semantic-Segmentation-PyTorch`, 2021.

[21] X. Zhang, J. Zou, K. He, and J. Sun. Accelerating very deep convolutional networks for classification and detection. *CoRR*, abs/1505.06798, 2015.

5. Appendix

	SegNet	Bayesian Segnet	Counts
Global accuracy	7.71E-01	7.74E-01	1.64E+07
Class average	2.63E-01	2.54E-01	-
Unlabeled	0.00E+00	0.00E+00	4.75E+03
Ego Vehicle	9.64E-01	9.63E-01	7.11E+05
Rectification Border	8.38E-01	7.72E-01	3.16E+05
Out of ROI	5.70E-01	5.35E-01	1.60E+05
Static	9.76E-02	8.85E-02	2.15E+05
Dynamic	1.98E-01	1.61E-01	1.39E+05
Ground	1.15E-01	5.70E-02	3.52E+05
Road	9.39E-01	9.45E-01	5.32E+06
Sidewalk	6.73E-01	7.44E-01	7.52E+05
Parking	6.57E-02	1.38E-03	1.27E+05
Rail Track	1.49E-02	9.38E-03	7.79E+04
Building	9.16E-01	9.34E-01	2.89E+06
Wall	7.43E-02	8.02E-02	2.15E+05
Fence	4.34E-02	1.91E-02	1.96E+05
Guard Rail	1.19E-05	0.00E+00	8.40E+04
Bridge	4.98E-05	3.73E-05	8.03E+04
Tunnel	1.30E-05	1.30E-05	7.70E+04
Pole	7.79E-05	1.01E-04	1.28E+05
Pole Group	6.35E-05	6.35E-05	7.88E+04
Traffic Light	9.69E-05	6.17E-05	1.14E+05
Traffic Sign	7.96E-03	2.07E-02	1.86E+05
Vegetation	9.08E-01	8.96E-01	2.27E+06
Terrain	3.25E-01	3.17E-01	1.64E+05
Sky	8.41E-01	8.51E-01	4.62E+05
Person	4.40E-01	3.07E-01	1.52E+05
Rider	7.61E-03	3.84E-02	5.98E+04
Car	8.90E-01	9.08E-01	8.35E+05
Truck	0.00E+00	0.00E+00	4.75E+04
Bus	0.00E+00	0.00E+00	5.47E+04
Caravan	0.00E+00	0.00E+00	6.06E+03
Trailer	0.00E+00	0.00E+00	8.37E+03
Train	0.00E+00	0.00E+00	2.01E+04
Motorcycle	0.00E+00	0.00E+00	1.50E+04
Bicycle	2.59E-03	4.49E-05	6.69E+04

Table 1: Quantitative results of SegNet and Bayesian SegNet on CityScapes. *Global accuracy* is the proportion of correctly guessed pixels overall. *Class average* is the average of the accuracies for each class. *Counts* is the number of pixels in each class. *Note: I apologize for the messiness of this table; I am coming up against time constraints.*